Python Viva Preparation Guide

18 Most Important Questions & Answers

Covering: Basics · Functions · OOP · File/Exception Handling · Data Structures

Topic	Count	Key Areas
Basics	5 Qs	Data types, loops, operators
Functions	4 Qs	def, *args, lambda, recursion
OOP	4 Qs	Classes, __init__, inheritance, pillars
File & Exception	3 Qs	open(), try/except, raise/assert
DS & Misc	5 Qs	list comp, dict, modules, GIL

Difficulty Legend: Easy | Medium | Hard

Basics

Q1 Easy

What is Python? Name some key features.

Answer:

• High-level, interpreted, general-purpose language.

• Interpreted – runs line by line, no compilation needed.

• Dynamically typed – no need to declare variable types.

• Indentation – uses indentation instead of braces for blocks.

• Object-oriented – supports classes and objects.

• Large standard library – 'batteries included'.

Q2 Easy

What is the difference between a list and a tuple?

Answer:

• list = mutable (can add/remove/change elements).

• tuple = immutable (cannot be modified after creation).

• Tuples are faster and used for fixed data like coordinates.

```
my_list = [1, 2, 3] # mutable
my_tuple = (1, 2, 3) # immutable
```

Q3 Easy

What is the difference between == and is?

Answer:

• == checks if the values are equal.

• is checks if both variables point to the same object in memory.

```
a = [1, 2]
b = [1, 2]
a == b # True (same values)
a is b # False (different objects)
```

Q4 Easy

What are Python built-in data types?

Answer:

• int – integers (5, -3)

• float – decimals (3.14)

• str – strings ('hello')

• bool – True or False

• list – ordered, mutable collection

• tuple – ordered, immutable

• dict – key-value pairs

• set – unordered unique items

Q5 Easy

What is the difference between break, continue, and pass?

Answer:

• break – exits the loop completely.

• continue – skips current iteration, moves to next.

• pass – does nothing; used as a placeholder.

```
for i in range(5):
if i == 3: break # stops at 3
if i == 1: continue # skips 1
pass # placeholder
```

Functions

Q6 Easy

What is a function? How do you define one?

Answer:

• A reusable block of code that performs a specific task.

• Defined with the def keyword followed by function name and parameters.

```
def greet(name):
return 'Hello, ' + name

print(greet('Aryan')) # Hello, Aryan
```

Q7 Medium

What are *args and **kwargs?

Answer:

• *args allows any number of positional arguments (stored as a tuple).

• **kwargs allows any number of keyword arguments (stored as a dict).

```
def show(*args, **kwargs):
print(args) # (1, 2, 3)
print(kwargs) # {'name': 'Aryan'}

show(1, 2, 3, name='Aryan')
```

Q8 Medium

What is a lambda function?

Answer:

• An anonymous (nameless) function defined in a single line.

• Used when a small function is needed temporarily, e.g. inside map() or sorted().

```
square = lambda x: x * x
print(square(5)) # 25
```

Q9 Medium

What is recursion? Give an example.

Answer:

• A function that calls itself to solve a smaller version of the same problem.

• Every recursive function needs a base case to stop the recursion.

```
def factorial(n):
if n == 0:
return 1
return n * factorial(n - 1)

print(factorial(5)) # 120
```

OOP

Q10 Easy

What is a class and an object?

Answer:

• A class is a blueprint/template.

• An object is an instance created from a class.

```
class Dog:
def __init__(self, name):
self.name = name
def bark(self):
print(self.name + ' says woof!')

d = Dog('Bruno') # d is an object
d.bark() # Bruno says woof!
```

Q11 Easy

What is the use of __init__?

Answer:

• __init__ is the constructor method.

• Automatically called when an object is created.

• Used to initialize the object's attributes.

```
class Student:
def __init__(self, name, marks):
self.name = name
self.marks = marks
```

Q12 Medium

What is inheritance? Give an example.

Answer:

• Allows a child class to reuse properties and methods from a parent class.

• Promotes code reusability and hierarchical relationships.

```
class Animal:
def speak(self):
print('Some sound')

class Cat(Animal):
def speak(self):
print('Meow')

c = Cat()
c.speak() # Meow
```

Q13 Medium

What are the 4 pillars of OOP?

Answer:

• Encapsulation – hiding internal data using private variables (self.__x).

• Abstraction – showing only essential details, hiding complexity.

• Inheritance – child class inherits from parent class.

• Polymorphism – same method name behaves differently in different classes.

File & Exception

Q14 Easy

How do you open and read a file in Python?

Answer:

• Use the open() function with a mode: 'r' read, 'w' write, 'a' append.

• Using with automatically closes the file when done.

```
with open('file.txt', 'r') as f:
content = f.read()
print(content)
```

Q15 Easy

What is exception handling? Write an example.

Answer:

• Prevents the program from crashing on runtime errors.

• try – risky code block.

• except – handles the error.

• finally – always runs, whether error occurred or not.

```
try:
x = int(input('Enter number: '))
print(10 / x)
except ZeroDivisionError:
print('Cannot divide by zero!')
except ValueError:
print('Invalid input!')
finally:
print('Done')
```

Q16 Hard

What is the difference between raise and assert?

Answer:

• raise – manually triggers an exception.

• assert – checks a condition; raises AssertionError if False.

• assert is mainly used for debugging/testing.

```
raise ValueError('Invalid age')

assert age > 0, 'Age must be positive'
```

DS & Misc

Q17 Medium

What is list comprehension? Give an example.

Answer:

• A concise way to create a list from another list or range.

• Much shorter than a traditional for loop.

```
# Normal way
squares = []
for i in range(5):
squares.append(i*i)

# List comprehension
squares = [i*i for i in range(5)]
# [0, 1, 4, 9, 16]
```

Q18 Easy

What is a dictionary? How do you access values?

Answer:

• Stores data as key-value pairs.

• Access values using square brackets or .get() method.

```
student = {'name': 'Aryan', 'marks': 85}

print(student['name']) # Aryan
print(student.get('marks')) # 85
student['marks'] = 90 # update
```

Q19 Easy

What is the difference between append() and extend()?

Answer:

• append() adds a single element to the end of the list.

• extend() adds all elements of another iterable.

```
a = [1, 2]
a.append([3, 4]) # [1, 2, [3, 4]]

b = [1, 2]
b.extend([3, 4]) # [1, 2, 3, 4]
```

Q20 Easy

What is a module? How do you import one?

Answer:

• A module is a Python file with reusable functions, classes, or variables.

• Use import keyword or from...import for specific names.

```
import math
print(math.sqrt(16)) # 4.0

from math import pi
print(pi) # 3.14159...
```

Q21 Hard

What is the Global Interpreter Lock (GIL)?

Answer:

• A mutex in CPython that allows only one thread to execute Python bytecode at a time.

• Even on multi-core CPUs, threads don't truly run in parallel for CPU-bound tasks.

• I/O-bound tasks (networking, file reading) still benefit from threads.

• Use the multiprocessing module to bypass the GIL for CPU-heavy work.